

TopDown

Ziel von **boostedriirt** ist es, iterativ einen „Boosted Random Intercept Regression Tree“ zu erzeugen. Dabei wird die Stärke von Regressionsbäumen bei Nichtlinearität mit einem Random-Intercept-Linear-Mixed-Model kombiniert, um gruppenspezifische Effekte abzubilden. Typische Anwendungsfälle sind Daten mit Clustern/Gruppen, in denen systematische Unterschiede vorherrschen, z. B. Schulen, Universitäten, Small-Area-Analysen oder Zeitreihen mit einem Cluster pro Subjekt. Das Paket nutzt hierfür eine 5-fache Cross-Validation (CV).

Wir starten mit einem Intercept β_0/μ_0 . Auf Basis dieses Intercepts berechnen wir den Gradienten (Residuen), den ein reines Intercept-Modell nicht erklärt, und trainieren darauf einen Regression Tree. Verbleibende, vermutlich gruppenbasierte Unterschiede werden über ein Random Intercept geschätzt. Beide Beiträge werden mit einer Lernrate kombiniert, um Overfitting zu vermeiden, und verbessern pro Iteration die Vorhersage, bis keine Verbesserung mehr messbar ist oder 100 Iterationen erreicht sind. Die Lernrate sorgt dafür, dass das Modell bei jeder Iteration leicht besser wird und nicht sofort den Fehler korrigiert. Die Verbesserung messen wir am RMSE. Das resultierende Modellobjekt kann über S3-Methoden genutzt werden.

TopDown Code:

1. Hauptfunktion:

```
# Inputs:
# Clustered Dataframe
# ID Gruppenvariable
# Y numeric (Outcome)
# p > 0 Kovariablen (numeric/integer)
# alpha = 0.1 learning rate
# iter_max = 100

# Output:
# S3_objekt: Random Intercept Regression Tree

Boostriirt_model <- function(grouped_df, ID) {
```

```

#Unterfunktionen

#Output
}

```

2. Unterfunktionen

```

#0. Input-Check
checking_inputs <- function(df, Y, ID, X) {
  # df ist data.frame
  # Y numerisch; X numerisch/integer;
  # ID vorhanden (faktor)
  # NAs?
}

#1. 5-fach CV
folds_id <- function(id_vec, k, seed) {
  return(folds)
}

#2. Parameter-Tuning
cv_parameters <- function(X, Y, ID, k, colsample, subsample, maxdepth, seed) {
  # (grouping = ID)
  # nutze CART-Proxy (maxdepth=3)
  # bestes Paar (colsample, subsample)
  output <- best_params
}

#3. Trainingsalgorithmus (CV)
train_cv_rirt <- function(X, Y, ID, folds, alpha, iter_max, colsample, subsample, maxdepth) {
  # Für jeden Fold:
  #   - Setze f0 = mean(Y_train)
  #   - für iter = iter_max oder keine Verbesserung:
  #       (a) Stichprobe (test/train) subsample, colsample
  #       (b) g_iter = Y_train - f_iter-1
  #       (c) CART auf g_iter mit maxdepth=3
  #       (d) e_iter = Y_train - (f_train + f_CART_train)
  #       (e) LMM: e_iter ~ f0 + (1 | ID)
  #       (f) Aktualisierung: f_new = f_old + alpha * ( f_CART + (f_LMM - f0) )
  #       (g) RMSE auf Fold berechnen
  # Stop if (iter = iter_max) oder (RMSE = RMSE-5)
}

```

```

    output <- iter_final
  }

#4. Finales Modell (Voller Datensatz)
# Wie schritt 3 aber auf vollen Daten und festen iter_final
Boostriert <- function(X, Y, ID, colsample, subsample, alpha, iter_final, maxdepth) {
  # initialisiere f0 = mean(Y)
  # für iter = iter_final:
  #   - subsample/colsample
  #   - g = Y - f
  #   - CART auf g
  #   - e = Y - (f + f_CART)
  #   - LMM auf e: e ~ 0 + (1 | ID)
  #   - Update: f <- f + alpha * ( f_CART + (f_LMM - 0) )
  output <- c(fitted_values, rmse_final)
}

#5. S3-Objekt Boostedriert bauen
make_boosted_riert <- function(final_fit, Boostriert_values) {

}

#6. S3-Checks
# print.boosted riert
# summary.boosted riert
# plot.boosted riert
# predict.boosted riert

```